# 1 Part 1 – Autonomous Systems Fundamentals

## 1.1 General

### Q1. What does it mean for a system to be autonomous?

An autonomous system can sense its environment, make decisions, and act without continuous human control.

The main modules are:

- **Perception:** Detects objects, obstacles, lanes, markers, and other environmental features.
- **Localization:** Estimates the robot's position and orientation.
- **Mapping:** Builds or uses a map of the environment.
- **Planning:** Decides the path or action needed to reach the goal.
- **Control:** Converts the planned motion into motor commands.

The typical data flow is:

$$Sensors \rightarrow Perception \rightarrow Localization/Mapping \rightarrow Planning \rightarrow Control \rightarrow Actuators$$

### Q2. Role of simulation and testing

Simulation allows developers to test an autonomous system before using real hardware. It helps in designing algorithms safely, testing many scenarios quickly, finding bugs early, and reducing risk before real-world deployment.

Simulation is useful because dangerous or rare situations can be tested without damaging hardware or causing safety risks.

### Q3. Limitations of simulation

Simulation is not always identical to reality. This can happen because physics models are simplified, friction and wheel slip may be inaccurate, collisions may not be perfectly modeled, and sensors may not include realistic noise.

Other differences include lighting, weather, timing delays, vibration, hardware drift, and unexpected failures. Because of this, a robot may work well in simulation but behave differently in the real world.

### Q4. Unrealistic sensor model in Gazebo

A sensor model is unrealistic when it does not behave like a real sensor.

Examples include incorrect noise, missing bias or drift, wrong update rate, ignored delay, and unrealistic interaction with lighting, reflections, or occlusion.

This can make the system perform better in simulation than it would on real hardware.

## 1.2 ROS 2

### Q1. ROS 2 concepts

A **node** is a program that performs a specific task, such as reading sensor data or controlling motors.

Nodes communicate using **topics**. A node that sends data is called a **publisher**, while a node that receives data is called a **subscriber**. The data being exchanged is called a **message**.

For example, a LiDAR node can publish scan data on a topic, and a navigation node can subscribe to that topic to use the data for obstacle avoidance.

A **launch file** is used to start multiple nodes together with their parameters, making it easier to run a complete robotic system.

### Q2. QoS policies in ROS 2

Quality of Service, or QoS, controls how ROS 2 handles communication between publishers and subscribers.

Important QoS settings include:

- **Reliability:** Determines whether messages must be guaranteed or can be dropped.
- **Durability:** Determines whether late subscribers can receive previously published messages.

If the publisher and subscriber QoS settings are incompatible, communication may fail even if the topic name and message type are correct.

## 1.3 Path Planning

### Q1. Define path planning

Path planning is the process of finding a feasible route from the robot's current position to a goal.

It considers obstacles, map information, robot size, motion limits, and kinematic constraints.

In a navigation stack, planning decides where the robot should go, while control decides how the robot should move to follow that plan.

### Q2. Global vs local path planners

A **global planner** creates a complete path from the start position to the goal using a map. It focuses on the overall route.

A **local planner** generates short-term motion commands while avoiding nearby obstacles in real time.

The global planner provides the main route, while the local planner follows that route and reacts to dynamic changes in the environment.

### Q3. Occupancy grid maps vs height maps

An **occupancy grid map** divides the environment into cells marked as free, occupied, or unknown.

Its advantages are that it is simple, useful for 2D navigation, and suitable for many indoor mobile robots. Its limitation is that it does not represent height information well.

A **height map** stores elevation information for each area.

Its advantages are that it is useful for uneven terrain and outdoor navigation. Its limitations are that it is more complex and may not fully represent overhangs or vertical structures.

### Q4. A* algorithm

A* is a graph search algorithm used to find the lowest-cost path.

It uses:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the cost from the start node to the current node.
- $h(n)$ is the estimated cost from the current node to the goal.
- $f(n)$ is the total estimated cost.

A* expands the node with the lowest $f(n)$. It guarantees an optimal path if the heuristic is admissible, meaning it never overestimates the true cost to the goal.

### Q5. Artificial Potential Field method

The Artificial Potential Field method treats the robot like a particle affected by forces.

The goal creates an attractive force, while obstacles create repulsive forces. The robot moves according to the combined force.

The main limitations are that the robot may get stuck in local minima, oscillate near obstacles, or struggle in narrow passages.

## 1.4 Control

### Q1. Define control theory

Control theory studies how to make physical systems behave in a desired way.

In robotics, it is used to control position, speed, orientation, motion, and stability.

### Q2. Open-loop vs closed-loop control

An **open-loop** control system acts without feedback. It does not know whether the desired result was achieved.

A **closed-loop** control system uses feedback to compare the actual output with the desired output and correct errors.

Closed-loop systems are usually more accurate and robust.

#### Q3. Controller stability

A controller is stable if the system does not grow uncontrollably or oscillate forever.

In practical terms, a stable robot reaches the desired position or motion smoothly and remains under control over time.

#### Q4. Pure Pursuit algorithm

Pure Pursuit is a path-tracking algorithm.

It selects a lookahead point on the path ahead of the robot. The robot then calculates a steering command to move toward that point.

The geometric principle is that the robot follows a circular arc from its current position to the lookahead point.

#### Q5. Standard vs Adaptive Pure Pursuit

Standard Pure Pursuit uses a fixed lookahead distance.

Adaptive Pure Pursuit changes the lookahead distance depending on speed, curvature, or tracking error.

Adaptation is useful because a larger lookahead gives smoother motion at high speed, while a smaller lookahead improves accuracy in sharp turns.

### 1.5 Localization

#### Q1. Sensor fusion

Sensor fusion combines data from multiple sensors to produce a better estimate than using one sensor alone.

It is necessary because every sensor has weaknesses. For example, IMUs drift, cameras need good lighting, and LiDAR can fail on reflective or transparent surfaces.

#### Q2. IMU structure and function

An IMU usually contains:

- **Accelerometer:** Measures linear acceleration.
- **Gyroscope:** Measures angular velocity.
- **Magnetometer:** Measures magnetic heading, if included.

In localization, IMU data helps estimate orientation, velocity, and motion.

However, IMUs suffer from noise and drift, especially when acceleration is integrated over time.

#### Q3. Kalman Filter and Extended Kalman Filter

A **Kalman Filter** estimates the state of a linear system using prediction and measurement updates.

An **Extended Kalman Filter** is used for nonlinear systems. It linearizes the system around the current estimate.

A Kalman Filter is used for linear models, while an EKF is used for nonlinear robotics problems such as robot localization.

### 1.6 Perception

#### Q1. LiDAR vs camera

LiDAR measures distance by sending laser pulses and measuring their return time. It produces laser scans or point clouds.

Cameras capture images using light. They produce 2D image data with color and texture information.

LiDAR gives direct distance measurements and is usually processed geometrically. Cameras provide richer visual information but often require computer vision or AI processing.

LiDAR can fail with glass, mirrors, rain, or reflective surfaces. Cameras can fail in poor lighting, glare, fog, or motion blur.

#### Q2. Monocular vs stereo cameras

A **monocular camera** uses one camera. It cannot directly measure depth without motion, learning, or assumptions.

A **stereo camera** uses two cameras and estimates depth from the difference between the two images.

Stereo cameras provide better depth estimation but require more calibration and computation.

**Q3. ArUco markers**

ArUco markers are square visual fiducial markers with unique black-and-white patterns.
They are detected by finding the square border in an image and decoding the internal pattern.
From a detected marker, the system can extract the marker ID, position, orientation, and camera-to-marker pose.
ArUco markers can be used for localization, camera calibration, object tracking, and robotic manipulation tasks.